

Go (Golang) Backend Learning Summary

Prepared By : Mansuri Adnan

1 Go Language Fundamentals

- Variables, constants (`const`), and Go's static type system (`int`, `float64`, `bool`, `string`)
- Control flow: `if/else`, `switch`, and Go's single `for` loop covering counting, conditional, and infinite loop patterns
- Slices and `range` iteration

2 Core Go — Functions, Structs, Interfaces & Errors

Functions & Structs

- Functions with single and multiple return values (`(int, error)` pattern)
- Structs as Go's data-modeling primitive; exported vs. unexported identifiers via capitalization
- Methods with value receivers (read-only) vs. pointer receivers (`func (t *Task) MarkDone()`) for mutation

Interfaces

- Implicit interface satisfaction — no `implements` keyword; any type with the required method set qualifies
- Built-in `Stringer` interface (`String() string`) for automatic custom formatting in `fmt` output

Custom Types & Error Handling

- Named types over primitives (`type Priority int`) for domain clarity and type safety
- Custom error types implementing the `error` interface, inspected with `errors.As`
- Error wrapping with `fmt.Errorf("...: %w", err)` to preserve context up the call stack

3 Pointers & Memory

- Address-of (`&`) and dereference (`*`) operators, and pass-by-value vs. pass-by-reference semantics
- Why pointer receivers are required for methods that mutate struct state
- Avoiding unnecessary struct copies on function calls

4 Concurrency

Goroutines & Synchronization

- `go` keyword for launching lightweight concurrent functions
- `sync.WaitGroup` for deterministic waiting on multiple goroutines (replacing unreliable `time.Sleep`)

- Channels (`chan`), buffered channels, `close()`, and `range` over a channel
- `select` for multiplexing across multiple channels, including non-blocking reads via `default`

Safe Shared State

- Race conditions demonstrated directly (a naive shared counter across 1000 goroutines) before introducing the fix
- `sync.Mutex` / `Lock()` / `Unlock()` to protect shared state

Worker Pools

- Bounded concurrency pattern: fixed pool of workers consuming from a shared job channel
- Benchmarked directly — 10 simulated jobs processed in ~4.0s with 1 worker vs. ~0.8s with 5 workers

5 HTTP Fundamentals & REST APIs (net/http)

- HTTP request/response anatomy: methods, headers, status codes, body
- Servers built directly on `net/http` — `http.HandleFunc`, `http.ResponseWriter`, `*http.Request`
- JSON marshaling/unmarshaling via `encoding/json` (`Encoder/Decoder`) with struct tags (`json:"field"`)
- Manually implemented middleware (function-wrapping-function pattern) for logging and auth checks

6 Gin Framework — REST APIs

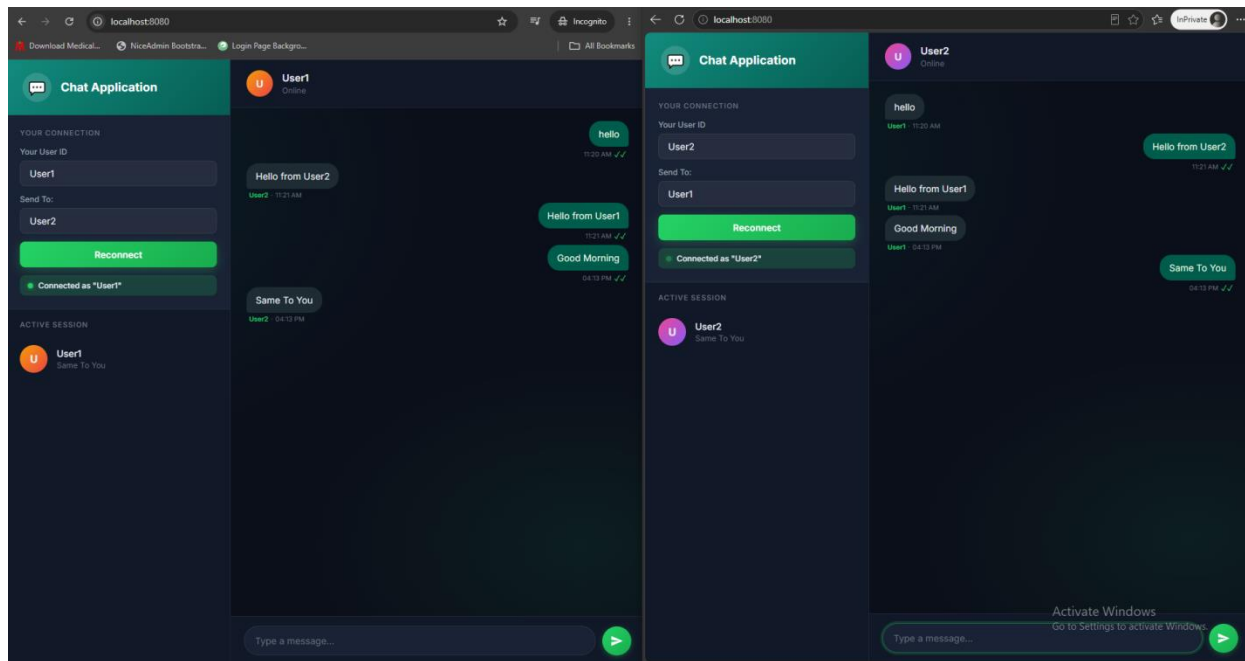
- Routing, `gin.Context`, and JSON responses (`c.JSON`) replacing manual `encoding/json` boilerplate
- Full CRUD implementation: `GET` / `POST` / `PUT` / `DELETE`, including correct status codes
- Gin-native middleware (`gin.HandlerFunc`, `c.Next()`, `c.Abort()`) and route groups for selectively protected endpoints
- Built-in `gin.Recovery()` / `gin.Logger()` middleware

7 Real-Time Communication — WebSockets

Implemented using `gorilla/websocket`, covering the full connection lifecycle from HTTP upgrade through multi-client broadcast.

- HTTP-to-WebSocket upgrade handshake via `websocket.Upgrader`
- Per-connection read loop (`conn.ReadMessage`) running on its own goroutine per client
- A concurrency-safe `Hub` (`map[*websocket.Conn]bool` + `sync.Mutex`) tracking all active connections
- Connect/disconnect lifecycle management with automatic cleanup on client drop

Working two-client chat demo — a message sent by User1 is broadcast instantly to User2 (and vice versa) over a live WebSocket connection.



8 Real-Time Communication — Server-Sent Events (SSE)

- One-way server-to-client event streaming via `text/event-stream` and `c.SSEvent` / `c.Stream`
- Explicit response flushing (`c.Writer.Flush()`) to guarantee incremental delivery rather than buffered dumps

9 Clean Architecture

- Strict layering: Handler → Service → Repository, with dependencies flowing one direction only
- `Repository` defined as an interface, decoupling business logic from the storage mechanism
- Business rules (e.g., duplicate-title rejection) isolated in the Service layer, independent of HTTP or storage concerns
- Dependency Injection wired explicitly in `main.go` (repository → service → handler)

10 Authentication & API Security

Password & Token Handling

- `bcrypt` password hashing (`GenerateFromPassword` / `CompareHashAndPassword`)
- JWT access tokens (`golang-jwt/jwt`) with signed claims and expiration (`exp`)

11 Proof-of-Concept Projects

1. Real-Time Chat Application — Go + Gin + WebSocket

- Multi-user chat backend built on Gin and `gorilla/websocket`, applying the Hub/broadcast pattern and end-to-end in a real application rather than an isolated exercise
- Each client upgraded from HTTP to a WebSocket connection, tracked in a concurrency-safe connection registry, with messages fanned out to all active participants in real time
- Verified with multiple simultaneous browser sessions — messages sent by one user appear instantly for all other connected users, with correct connect/disconnect handling

2. JWT Authentication POC

- Standalone authentication service implementing signup and login endpoints with `bcrypt`-hashed password storage
- JWT access-token issuance on successful login, with a dedicated verification middleware validating the `Authorization: Bearer <token>` header on protected routes
- Confirmed the full auth lifecycle: signup → login → access denied without a token → access granted with a valid token → access denied with a tampered/expired token